0.1 Constructing the optimal level schedule

Source items 6.6 and 10.3; module GraphSchedules

Let V be finite and let $u \to v$ mean that task v depends on task u. Assume there is no nonempty directed cycle. A topological order or valid level function is *not* supplied as an additional hypothesis.

For each vertex, define its strict ancestor set

$$\mathrm{Anc}(v) = \{u : \text{there is a positive-length path } u \rightsquigarrow v\}.$$

For every edge $u \to v$, the proof establishes

$$\mathrm{Anc}(u) \subsetneq \mathrm{Anc}(v).$$

The inclusion follows by appending the edge. It is strict because u is an ancestor of v but cannot be its own strict ancestor in an acyclic graph. Cardinality therefore decreases when following predecessors. This constructs a well-founded relation on which the level recursion is defined:

$$\boxed{\ell(v) = 1 + \max_{u \to v} \ell(u), \qquad \max \emptyset = 0.}$$

Validity and attainment. Lean proves $\ell(v) > 0$ and $\ell(u) < \ell(v)$ on every edge. A path of n edges from a to b satisfies

$$\ell(a) + n \le \ell(b).$$

Conversely, predecessor-max attainment and induction produce an actual path with $\ell(v) - 1$ edges ending at each vertex v. The result is an equality with a longest path, not merely an upper bound.

A valid schedule $s : V \to \mathbb{N}$ assigns positive stages and strictly increases on each edge. Induction proves the stronger pointwise statement

$$\ell(v) \le s(v) \quad \text{for every } v.$$

Consequently the constructed schedule achieves the least possible makespan,

$$\boxed{d = \max_{v \in V} \ell(v).}$$

For a nonempty graph, d equals the number of vertices on an attained longest path. Vertices of the same level have no dependency edge in either direction.

Empty graph and interpretation. When $V = \emptyset$, the checked depth is zero. The phrase "one plus the longest edge-path length" applies to nonempty graphs. These are ideal discrete scheduling statements. Weighted task costs, a finite processor budget, launch latency and communication are not part of this model. The definitions are mathematical constructions, not a certified hardware scheduler.